

Q1: How many entries in your database are from applicants who applied for Fall 2026?

Result:

Applicant count: 32544

SQL Query:

```
SELECT COUNT(*) AS cnt_fall26
FROM applicants
WHERE term = 'Fall 2026'
```

Explanation:

Filters to term equal to 'Fall 2026', then counts those rows.

Q2: Among entries that provide a nationality classification, what percentage are international students?

Result:

International percentage: 46.31%

SQL Query:

```
SELECT
    ROUND(
        SUM(
            CASE WHEN us_or_international = 'International' THEN 1
            ELSE 0
            END
        ) * 100.0 / COUNT(us_or_international), 2
    ) || '%' AS pct_intl
FROM applicants
WHERE us_or_international IS NOT NULL
```

Explanation:

Restricts to rows with a known nationality classification (which should be all of them anyways since us_or_international has NOT NULL schema), counts how many are 'International' out of that total, and expresses it as a percentage.

Q3: What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

Result:

Average GPA: 3.76, Average GRE: 165.67, Average GRE V: 160.41, Average GRE AW: 4.32

SQL Query:

```
WITH avg_gpa AS (  
    SELECT ROUND(AVG(gpa)::numeric, 2) as avg_gpa  
    FROM applicants  
    WHERE gpa IS NOT NULL  
)  
,  
avg_gre AS (  
    SELECT ROUND(AVG(gre)::numeric, 2) as avg_gre  
    FROM applicants  
    WHERE gre IS NOT NULL  
)  
,  
avg_gre_v AS (  
    SELECT ROUND(AVG(gre_v)::numeric, 2) as avg_gre_v  
    FROM applicants  
    WHERE gre_v IS NOT NULL  
)  
,  
avg_gre_aw AS (  
    SELECT ROUND(AVG(gre_aw)::numeric, 2) as avg_gre_aw  
    FROM applicants  
    WHERE gre_aw IS NOT NULL  
)  
SELECT *  
FROM avg_gpa, avg_gre, avg_gre_v, avg_gre_aw
```

Explanation:

Computes the average of each metric separately, each over only the rows that actually report that metric, then combines the four single-row results into one row so all four averages answer the question together.

Q4: What is the average GPA of American applicants who applied for Fall 2026?

Result:

Average GPA American: 3.79

SQL Query:

```
SELECT ROUND(AVG(gpa)::numeric, 2) as avg_gpa_us_fall26  
FROM applicants  
WHERE gpa IS NOT NULL  
    AND us_or_international = 'American'  
    AND term = 'Fall 2026'
```

Explanation:

Filters to rows that are American, applied for Fall 2026, and actually report a GPA before reporting average GPA.

Q5: What percentage of Fall 2025 entries are acceptances?

Result:

Percentage accepted: 39.87%

SQL Query:

```
SELECT
    ROUND(
        SUM(
            CASE WHEN status LIKE 'Accepted%' THEN 1
            ELSE 0
            END
        ) * 100.0 / COUNT(status), 2
    ) || '%' AS pct_accepted_fall25
FROM applicants
WHERE term = 'Fall 2025'
```

Explanation:

Filters to Fall 2025 entries, counts how many have a status starting with 'Accepted', and expresses that as a percentage of all Fall 2025 entries.

Q6: What is the average GPA of accepted applicants who applied for Fall 2026?

Result:

Average GPA accepted: 3.76

SQL Query:

```
SELECT
    ROUND(AVG(gpa)::numeric, 2) avg_gpa_accepted_fall26
FROM applicants
WHERE term = 'Fall 2026'
    AND gpa IS NOT NULL
    AND status LIKE 'Accepted%'
```

Explanation:

Filters to rows with reported GPA, where 'Accepted' is found in status, and term is 'Fall 2026'. Then, it averages GPA over rows those rows.

Q7: How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

Result:

JHU Masters Computer Science count: 18

SQL Query:

```
SELECT COUNT(*) AS cnt_jhu_ms_cs
FROM applicants
WHERE degree = 'Masters'
      AND program LIKE 'Computer Science, %'
      AND (program LIKE '%, Johns Hopkins University%'
           OR program LIKE '%, JHU%')
```

Explanation:

Filters to Masters degrees, programs which are 'Computer Science', and schools which are 'Johns Hopkins University'/'JHU'. Then, it counts those rows.

Q8: How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at one of the following universities? Georgetown University, Massachusetts Institute of Technology / MIT, Stanford University, Carnegie Mellon University

Result:

PhD Computer Science at listed schools: 11

SQL Query:

```
SELECT COUNT(*) AS cnt_fall26_cs_phd
FROM applicants
WHERE term = 'Fall 2026'
      AND degree = 'PhD'
      AND status LIKE 'Accepted%'
      AND program LIKE 'Computer Science, %'
      AND (program LIKE '%, Georgetown University%'
           OR program LIKE '%, Massachusetts Institute of Technology%'
           OR program LIKE '%, MIT%'
           OR program LIKE '%, Stanford University%'
           OR program LIKE '%, Carnegie Mellon University%')
```

Explanation:

Filters to results which are accepted PhDs in 'Computer Science' programs at 'Georgetown University', 'Massachusetts Institute of Technology'/'MIT', 'Stanford University', or 'Carnegie Mellon University'. Then, it counts all of those results.

Q9: Repeat Question 8, but identify the university and program using llm_generated_program and llm_generated_university.

Result:

PhD Computer Science at listed schools (LLM): 14, Difference: 3

SQL Query:

```
WITH q8 AS (
  SELECT COUNT(*) AS cnt
```

```

FROM applicants
WHERE term = 'Fall 2026'
      AND degree = 'PhD'
      AND status LIKE 'Accepted%'
      AND program LIKE 'Computer Science, %'
      AND (program LIKE '%, Georgetown University%'
           OR program LIKE '%, Massachusetts Institute of Technology%'
           OR program LIKE '%, MIT%'
           OR program LIKE '%, Stanford University%'
           OR program LIKE '%, Carnegie Mellon University%')
),
q9 AS (
  SELECT COUNT(*) AS cnt
  FROM applicants
  WHERE term = 'Fall 2026'
        AND degree = 'PhD'
        AND status LIKE 'Accepted%'
        AND llm_generated_program = 'Computer Science'
        AND (llm_generated_university LIKE 'Georgetown University%'
             OR llm_generated_university LIKE 'Massachusetts Institute of
             Technology%'
             OR llm_generated_university LIKE 'MIT%'
             OR llm_generated_university LIKE 'Stanford University%'
             OR llm_generated_university LIKE 'Carnegie Mellon University%')
)
SELECT q8.cnt AS cnt_fall26_cs_phd_original,
       q9.cnt AS cnt_fall26_cs_phd_llm,
       (q9.cnt - q8.cnt) AS llm_less_original
FROM q8, q9

```

Explanation:

Repeats Q8's count and Q8's count using the LLM-standardized program and university fields instead of the raw program text. Then, it reports both counts side by side with their difference, showing how much the standardization changes the result.

Why there is a difference:

The raw program field is free text typed by each poster, and the Q8 query matches it with LIKE patterns expecting a fairly specific format ("Computer Science, <exact university name>"). From my own testing using the '%' wildcard, the difference exists because of extra school names which are appended to the end of the university name. The LLM-generated fields collapse all of that free-text variation into one canonical spelling per university and program before this query ever runs, so entries that Q8's raw-text matching missed are correctly captured once standardized. That's why the LLM-based count (14) comes out higher than the raw-field count (11).

A1: What percentage of total acceptances come from each term found in the data?

Result:

Fall 2024 acceptance: 0.22%, Spring 2025 acceptance: 0.21%, Fall 2025 acceptance: 18.92%, Spring 2026 acceptance: 1.24%, Fall 2026 acceptance: 79.27%, Spring 2027 acceptance: 0.14%

SQL Query:

```
WITH accepted AS (  
    SELECT term, COUNT(*) AS cnt  
    FROM applicants  
    WHERE status LIKE 'Accepted%'  
    GROUP BY term  
)  
total_accepted AS (  
    SELECT COUNT(*) AS cnt  
    FROM applicants  
    WHERE status LIKE 'Accepted%'  
)  
SELECT accepted.term,  
    ROUND(accepted.cnt * 100.0 / total_accepted.cnt, 2) || '%' AS  
    pct_of_acceptances  
FROM accepted, total_accepted  
ORDER BY split_part(accepted.term, ' ', 2)::int,  
    CASE WHEN accepted.term LIKE 'Spring%' THEN 0 ELSE 1 END
```

Explanation:

Counts acceptances per term and the total acceptance count across all terms, divides each term's count by the total to get its share of all acceptances, and orders the terms in chronological order rather than alphabetically.

A2: What is the average GPA for students accepted to University of Southern California / USC, the average GPA for students not accepted there, and the difference in GPA?

Result:

USC average GPA accepted: 3.67, USC average GPA not accepted: 3.74, Difference: -0.07

SQL Query:

```
WITH usc AS (  
    SELECT gpa, status  
    FROM applicants  
    WHERE gpa IS NOT NULL  
    AND (program LIKE '%, University of Southern California%'
```

```

        OR program LIKE '%, USC%')
    ),
    accepted_avg AS (
        SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa
        FROM usc
        WHERE status LIKE 'Accepted%'
    ),
    not_accepted_avg AS (
        SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa
        FROM usc
        WHERE status NOT LIKE 'Accepted%'
    )
    SELECT accepted_avg.avg_gpa AS avg_gpa_accepted_usc,
           not_accepted_avg.avg_gpa AS avg_gpa_not_accepted_usc,
           (accepted_avg.avg_gpa - not_accepted_avg.avg_gpa) AS gpa_diff
    FROM accepted_avg, not_accepted_avg

```

Explanation:

Narrows to USC applicants with a reported GPA, splits them into accepted and not-accepted groups, averages GPA within each group, and subtracts the two averages.